

1 · Registering canonicalizations

```
def List_MapOp : List_Op<"map", [...]> {  
  ...  
  let hasCanonicalizer = 1;  
  ...  
}  
  
// ListOps.cpp  
void MapOp::getCanonicalizationPatterns(RewritePatternSet &results,  
                                        MLIRContext *context) {  
  results.add<YourRewritePatternsHere, NextPattern>(context);  
}
```

In ODS What you implement

hasCanonicalizer = 1	static void getCanonicalizationPatterns(RewritePatternSet &, MLIRContext *) - any number of patterns rooted at this op
hasFolder = 1	fold - no extra registration needed, the canonicalizer runs folders as well

2 · Is my pattern a canonicalization?

Canonical IR: “all programs that compute the same thing should have the same IR”. Which patterns get you there is a **design choice** – these are guidelines, not rules.

Yes

constant folding	arith.addi 1, 0 → 1
fewer operations	addi(%x, muli(%y, -1)) → subi(%x, %y)
it converges	no ping-pong with another pattern

No

information is lost litmus test: **“can I no longer perform an optimization afterwards?”** Then it is a **lowering** – put it in your own pass

Things you do *not* need to care about here: the performance of the generated code. Canonicalization is about form, not speed.

3 · Enabling fold for your Operation

```
def List_LengthOp : List_Op<"length", [Pure]> {  
  ...  
  // Whether this op has a folder.  
  let hasFolder = 1;  
}
```

Single-result op

```
OpFoldResult MyOp::fold(FoldAdaptor adaptor) {  
  ...  
  return nullptr;    // on failure  
}
```

Otherwise

```
LogicalResult MyOp::fold(FoldAdaptor adaptor,  
                        SmallVectorImpl<OpFoldResult> &results) {  
  ...  
  return failure();    // on failure (duh)  
}
```

A folder may **not** create or erase operations. It may inspect the surrounding IR.

4 · What fold returns: OpFoldResult

```
class OpFoldResult : public PointerUnion<Value, Attribute>
```

A Value replaces the op with an already existing value:

```
OpFoldResult ReverseOp::fold(FoldAdaptor adaptor) {  
  // list.reverse(list.reverse(%li)) -> %li  
  if (auto producer = getInput().getDefiningOp<ReverseOp>())  
    return producer.getInput();  
  return nullptr;  
}
```

An Attribute replaces the op with a constant:

```
%lb = arith.constant 1 : i32  
%ub = arith.constant 4 : i32  
%li = list.range %lb to %ub : !list.list<i32>  
// becomes  
%li = list.constant #list.list<[1, 2, 3]> : !list.list<i32>  
{}/ / nullptr means “did not fold”.
```

5 · The FoldAdaptor parameter

ODS generates it next to the op, as a mirror for folding: every operand getter returns that operand’s **constant value** instead of its Value, and is null when the operand is not a constant.

```
class PushBackOp : mlir::Op<...> {    class PushBackOp::FoldAdaptor {  
public:  
  Value getInput();                    Attribute getInput();    // null if not  
  Value getItem();                    Attribute getItem();    // a constant  
};                                    };
```

Combine it with dyn_cast_if_present – plain dyn_cast asserts on null:

```
OpFoldResult PushBackOp::fold(FoldAdaptor adaptor) {  
  auto constant = dyn_cast_if_present<ListAttr>(adaptor.getInput());  
  auto item = dyn_cast_if_present<IntegerAttr>(adaptor.getItem());  
  if (!constant || !item)  
    return {};  
  SmallVector<int64_t> elements(constant.getElements());  
  elements.push_back(item.getValue().getSExtValue());  
  return ListAttr::get(constant.getType(), elements);  
}
```

Attributes of the op are always constant, so read them from adaptor too.

6 · Constant-folding checklist

1. Have an attribute that can represent a constant of the result type.

MLIR C++ type	MLIR C++ attribute
---------------	--------------------

IntegerType	IntegerAttr
IntegerType (i1)	BoolAttr
list::ListType	list::ListAttr

2. Have a ConstantLike operation that holds it:

```
def List_ConstantOp : List_Op<"constant",  
  [ConstantLike, Pure, AllTypesMatch<["value", "result"]>]> {  
  let arguments = (ins List_ListAttr:$value);  
  let results = (outs List_ListType:$result);  
  let hasFolder = 1;    // fold() { return getValueAttr(); }  
}
```

3. Let the dialect materialize constants (next card).

7 · The dialect materializes constants

```
def List_Dialect : Dialect {  
  ...  
  let hasConstantMaterializer = 1;  
  // materializeConstant builds arith ops, so that dialect must be loaded too  
  let dependentDialects = [":mlir::arith::ArithDialect"];  
}
```

```
Operation *ListDialect::materializeConstant(OpBuilder &builder,  
                                           Attribute value, Type type,  
                                           Location loc) {  
  if (auto listValue = dyn_cast<ListAttr>(value)) {  
    if (listValue.getType() != type)  
      return nullptr;  
    return ConstantOp::create(builder, loc, listValue.getType(), listValue);  
  }  
  // Some folders produce integers or booleans instead!  
  return arith::ConstantOp::materialize(builder, value, type, loc);  
}
```

A fold whose attribute cannot be materialized is **silently dropped** – if your folder seems to do nothing, look here first.

8 · fold or RewritePattern?

Use fold	Use a RewritePattern
when you can – rule of least power	when you need to create operations
constant folding; FoldAdaptor is your friend	PatternRewriter is your friend
local: only this op’s result changes	may rewrite anything, anywhere
free in canonicalize, createOrFold and dataflow analyses	also fine for lowerings and opinionated transformations
canonicalization only	canonicalization or your own pass